

Take-Home Assessment

Project: Talk to a Document or YouTube Video

Objective

Build a **mobile-first web application** that allows users to interact with uploaded documents or YouTube videos through **live, voice-based conversations** powered by OpenAI's **Realtime API**.

The application will:

- Accept both:
 - A **YouTube URL** (with accessible transcript or captions), or
 - A **PDF** file (≤ 25 MB)
- Extract the text from the provided source
- Use that extracted text to **prime** a conversational context
- Initiate a **live voice conversation** with OpenAI's Realtime API, allowing the user to ask spoken questions and receive spoken answers in real time

The finished product should be intuitive, responsive, and functional across mobile devices.

Requirements

Functional

- **Ingestion**

Two sources must both work:

- A PDF upload
- A YouTube URL, using the video's transcript or captions

PDF ingestion must work in your deployed application.

YouTube ingestion must also work, but a local demonstration is acceptable for it. There is no official way to read captions from a video you do not own, so

most approaches depend on an unofficial endpoint that YouTube blocks from cloud provider IP ranges. If your deployed app cannot fetch transcripts for that reason, demo the YouTube path running locally and explain the limitation in your README.

Getting YouTube ingestion working on the deployed app as well is a bonus. If you manage it, tell us how.

Display the extracted text to the user in a collapsible or preview format.

- **Context Preparation**

- Once text is extracted, it should be made available for use as context in an OpenAI Realtime session.
- No chunking, summarization, or citation handling is required.

- **Voice Conversation**

- Include a “Start Voice Chat” button to establish a **live session** with the OpenAI Realtime Voice API.
- The user can **speak** questions and **hear** answers.
- Display a text transcript of the ongoing conversation in real time.
- If a microphone is unavailable, allow fallback to a **text-based** chat mode.

- **User Interface**

- Simple, mobile-first design optimized for ~390 px width screens.
- Include clear microphone and session controls (start, stop, mute, etc.).
- Provide visual connection status and conversation flow.

- **Security**

- API keys must **never** be exposed in the client.
- All Realtime connections must use **ephemeral tokens** or authenticated backend routes.

- **README**

- Provide setup and run instructions.
- Include a brief technical overview and any trade-offs made during development.

Guardrails & Constraints

- **Recommended Tech Stack (Not Mandatory)**

- **Frontend:** Next.js (App Router) + React + TypeScript
 - **Backend:** Node.js / Next API routes (or Express/Fastify) + TypeScript
 - **Realtime Voice Integration**
 - Use **OpenAI Realtime API** (WebRTC) for live audio sessions.
 - Issue **ephemeral tokens** from your backend for authentication.
 - Handle both microphone input and model audio output.
 - **PDF Extraction**
 - Perform server-side extraction (e.g., `pdf-parse`, `pdfjs-dist`, or equivalent).
 - **YouTube Transcript Retrieval**
 - Fetch transcripts server-side. Note that the YouTube Data API only returns captions for videos the authenticated account owns, so it will not work for arbitrary videos.
 - **Data Storage**
 - In-memory or lightweight database (e.g., SQLite) is sufficient.
 - No persistent user authentication required.
-

Deliverables

1. Working Web Application

- Frontend interface for uploading or entering a YouTube URL.
- Backend to handle ingestion, text extraction, and Realtime session setup.
- Hosted publicly (e.g., Vercel, Render, Railway, Firebase, Netlify, etc.) and ready to demo.

2. Video Walkthrough (10 – 15 minutes)

- Demonstrate the full workflow:
 - Upload or enter a YouTube link
 - View extracted text
 - Start a live voice conversation
- Explain key architectural and design decisions, including:
 - Framework choices and rationale

- Backend design and hosting
- OpenAI Realtime integration details
- Discuss any libraries, tools, or accelerators used.

3. GitHub Repository

- Publicly accessible repo with:
 - Clean and modular source code for frontend + backend
 - A README with setup instructions and environment configuration details
 - Any required dependencies (e.g., PDF libraries, OpenAI SDK)
 - Unit or integration tests where appropriate

Evaluation Criteria

Area	4 – Exceptional	3 – Strong	2 – Adequate	1 – Gaps
Product Polish (Mobile)	Delightful UX, robust states, responsive layout	Solid mobile layout and states	Functional on mobile, minor rough edges	Poor layout or inconsistent controls
Ingestion Quality	Handles edge cases for YouTube/PDF parsing	Reliable text extraction	Works on basic inputs	Fragile or frequent failures
Code Quality	Modular, typed, secure, well-documented	Clean, mostly typed, some testing	Understandable, minimal testing	Unstructured or insecure code
System Design	Clear separation, scalable architecture	Reasonable structure	Basic separation of concerns	Tightly coupled or ad-hoc design

Deliverable Deadline

Please submit your completed assignment within **7 business days** of receiving this prompt. If you need more time, just let us know — flexibility is fine.

Notes on AI Use

If you use AI-assisted tools (e.g., ChatGPT, Claude Code, Cursor), please briefly note where and how they helped. We're evaluating your **workflow and reasoning**, not penalizing tool usage.

Review Script (for us)

When reviewing your submission:

1. Paste a YouTube URL with available captions; confirm transcript ingestion.
2. Upload a PDF and confirm full text extraction.
3. Test on mobile: start a voice conversation, interrupt, follow up naturally.
4. Test under poor network conditions to assess stability.
5. Review code structure, environment handling, and secret management.